PML

UNIT I: Notes

(Note: Python Programming Language has been used for codes and Google Colab/Jupyter/python installed on system as interfaces)

Getting started with Tensorflow:

Introduction to Tensorflow, Ensuring that TensorFlow works, Representing tensors, Tensor types, Creating operators, Executing operators with sessions, Understanding code as a graph, Setting session configurations, Writing code in Jupyter, Using variables, Saving and loading variables, Visualizing data using TensorBoard, Overview of Keras, Architecture of Keras.

I. Introduction to TensorFlow

What is TensorFlow?

- **TensorFlow** is an open-source machine learning framework developed by Google.
- It allows developers to build, train, and deploy machine learning models.
- TensorFlow supports a wide range of machine learning and deep learning algorithms.
- Initially released in 2015, TensorFlow has become one of the most popular frameworks for machine learning.

Key Features of TensorFlow:

1. **Flexibility and Scalability:**
    - TensorFlow can be used for a variety of tasks, from simple linear regression to complex neural networks.
    - It supports both CPU and GPU computations, allowing for scalable and efficient processing.
2. **Eager Execution:**
    - TensorFlow 2.x introduced eager execution, which allows operations to be executed immediately as they are called from Python.
    - This makes the development and debugging process more intuitive and faster.
3. **Comprehensive Ecosystem:**
    - TensorFlow provides a comprehensive ecosystem of tools and libraries for various aspects of machine learning, including:
        - **TensorBoard:** For visualizing the training process and debugging.
        - **TensorFlow Lite:** For deploying models on mobile and edge devices.
        - **TensorFlow.js:** For running models in the browser using JavaScript.

- **TensorFlow Extended (TFX):** For end-to-end machine learning pipelines.

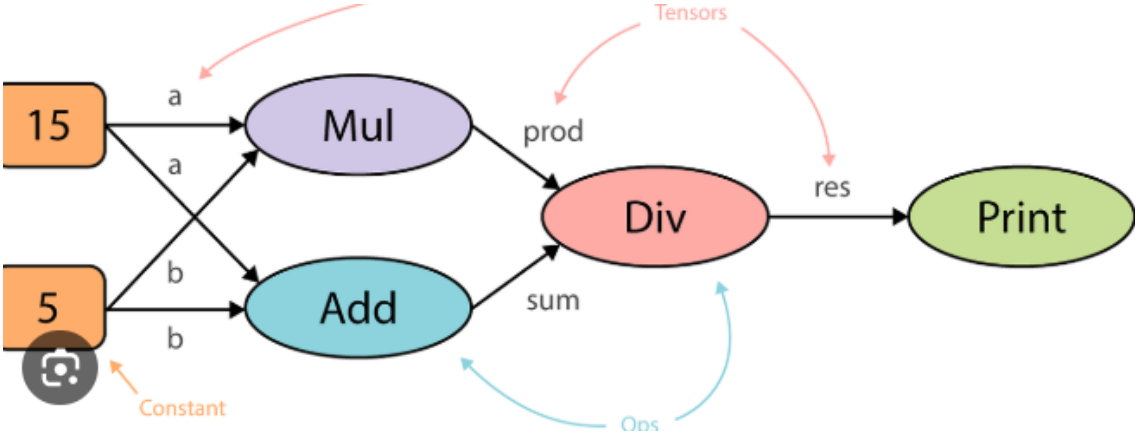
4. **Wide Community and Extensive Documentation:**
    - TensorFlow has a large and active community that contributes to its development.
    - Extensive documentation, tutorials, and pre-trained models are available to help users get started.

Applications of TensorFlow:

1. **Image Recognition:**
    - TensorFlow is widely used for image recognition tasks such as object detection, image classification, and image segmentation.
    - Pre-trained models like Inception and MobileNet are available for these tasks.
2. **Natural Language Processing (NLP):**
    - TensorFlow provides tools for NLP tasks such as sentiment analysis, language translation, and text generation.
    - Libraries like TensorFlow Text and BERT (Bidirectional Encoder Representations from Transformers) are used in NLP.
3. **Time Series Analysis:**
    - TensorFlow is used for time series forecasting, anomaly detection, and other time-based data analysis.
    - Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are commonly used for these tasks.
4. **Reinforcement Learning:**
    - TensorFlow supports reinforcement learning algorithms for training agents to perform tasks through trial and error.
    - Libraries like TensorFlow Agents provide tools for building and training reinforcement learning models.

TensorFlow Architecture:

1. **Computation Graphs:**
    - TensorFlow uses computation graphs to represent mathematical operations.
    - A computation graph is a series of TensorFlow operations arranged into a graph of nodes.
    - Nodes represent mathematical operations, while edges represent the data (tensors) flowing between them.



2. **Sessions (TensorFlow 1.x):**
   - In TensorFlow 1.x, sessions are used to execute operations in the computation graph.
   - A session is responsible for allocating resources (such as GPU or CPU) and running the computation graph.

   Example:

```
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()

a = tf.constant(5)
b = tf.constant(3)
add_op = tf.add(a, b)

with tf.Session() as sess:
    result = sess.run(add_op)
    print(result)  # Output: 8
```

3. **Eager Execution (TensorFlow 2.x):**
   - Eager execution allows operations to be executed immediately as they are called from Python.
   - This mode is more intuitive and easier to debug compared to the session-based execution in TensorFlow 1.x.

   Example:

```
import tensorflow as tf
```

```
a = tf.constant(5)
b = tf.constant(3)
add_result = tf.add(a, b)
print(add_result.numpy())  # Output: 8
```

Getting Started with TensorFlow:

1. **Installation:**

   TensorFlow can be installed using pip:pip install tensorflow

   It can also be installed using Anaconda:conda install tensorflow

2. **Basic Example:**

   Here's a basic example to get started with TensorFlow:

```
import tensorflow as tf

# Define a constant tensor
hello = tf.constant('Hello, TensorFlow!')

# Print the tensor
print(hello)
```

II. Ensuring that TensorFlow Works

Step-by-Step Guide to Ensure TensorFlow is Installed and Working

To start using TensorFlow, it's crucial to ensure that it is correctly installed and working on your system. Follow these steps to verify your TensorFlow setup.

I. Installing TensorFlow

Using pip:

1. **Open a Terminal or Command Prompt:**
   - On Windows: Press Win + R, type cmd, and press Enter.
   - On macOS/Linux: Open the Terminal from the Applications menu.
2. **Install TensorFlow:**
   - For the CPU version of TensorFlow (suitable for most beginner tasks):!pip install tensorflow
   - For the GPU version of TensorFlow (requires a compatible NVIDIA GPU and CUDA toolkit):!pip install tensorflow-gpu
3. **Using Anaconda:**

1. **Open Anaconda Prompt or Terminal:**
    - On Windows: Search for "Anaconda Prompt" in the Start menu and open it.
    - On macOS/Linux: Open the Terminal.

    Create a new Conda environment (optional but recommended):
    conda create --name tf_env python=3.8

    conda activate tf_env

    Install TensorFlow:conda install tensorflow

II. Verifying the Installation

Once TensorFlow is installed, you need to verify that it works correctly.

Verifying with a Python Script:

1. **Open a Python interpreter or a Jupyter Notebook.**
    - In the terminal or command prompt, type python and press Enter to open the Python interpreter.
    - Alternatively, you can open a Jupyter Notebook by typing jupyter notebook in the terminal.

    Run the following code:
    import tensorflow as tf

    print(tf.__version__)
2. **Check the Output:**
    - If TensorFlow is installed correctly, this code will print the TensorFlow version number (e.g., 2.5.0).

Verifying with a Simple TensorFlow Program:

Write a simple TensorFlow program to create a tensor and print its value.

```
import tensorflow as tf


# Create a constant tensor
hello = tf.constant('Hello, TensorFlow!')


# Print the tensor's value
print(hello)
```

1. **Run the Program:**
    - If you are using a script, save it as test_tf.py and run it using python test_tf.py.
    - If you are using a colab/Jupyter Notebook, simply run the cell.
2. **Check the Output:**

- The output should display a tensor object, such as tf.Tensor(b'Hello, TensorFlow!', shape=(), dtype=string).

Using TensorFlow in Google Colab:

1. **Open Google Colab:**
   - Go to Google Colab in your web browser.

   Create a new notebook and run the following code:

   ```
   import tensorflow as tf

   print(tf.__version__)
   ```

2. **Check the Output:**
   - The version number of TensorFlow should be printed, confirming that TensorFlow is working in the Colab environment.

Troubleshooting Common Issues

If you encounter issues during the installation or verification process, consider the following troubleshooting tips:

1. **Check Python Version:**
   - Ensure you are using a compatible Python version (TensorFlow generally supports Python 3.6-3.8).
2. **Upgrade pip:**

   Make sure your pip is up to date by running:pip install --upgrade pip
3. **Virtual Environment:**
   - Use a virtual environment to avoid conflicts with other packages.
   - Create a new environment using venv or conda.
4. **Error Messages:**
   - Read error messages carefully to understand the cause.
   - Common issues may involve incompatible packages, missing dependencies, or hardware requirements for GPU support.
5. **Consult Documentation:**
   - Refer to the TensorFlow installation guide for detailed instructions and troubleshooting.

III.Tensor Types and Creating Operators

Understanding Tensors

Tensors are the core data structures used in TensorFlow. They represent the data that flows through the computational graph.

1. **Definition:**

- A tensor is a multi-dimensional array with a uniform type (called a dtype).

2. **Rank (Order):**
   - The rank of a tensor is the number of dimensions or axes it has. For example:
     - Scalar (rank 0): A single number.
     - Vector (rank 1): A 1D array of numbers.
     - Matrix (rank 2): A 2D array of numbers.
     - Higher ranks: 3D, 4D, 5D tensors, etc.
3. **Shape:**
   - The shape of a tensor is a tuple of integers indicating the size of the tensor along each dimension.
   - Example: A tensor with shape (2, 3) has 2 rows and 3 columns.
4. **Data Types (dtypes):**
   - Tensors have a specific data type, such as tf.float32, tf.int32, tf.bool, etc.
   - The dtype specifies the type of data the tensor holds.

Creating Tensors

Tensors can be created in several ways using TensorFlow.

1. **Constant Tensors:**
   - Created with fixed values that do not change.

```
import tensorflow as tf

scalar = tf.constant(3)  # Scalar

vector = tf.constant([1, 2, 3])  # Vector

matrix = tf.constant([[1, 2], [3, 4]])  # Matrix
```

2. **Variable Tensors:**
   - Mutable tensors that can be updated.

```
variable = tf.Variable([1, 2, 3])
```

3. **Random Tensors:**
   - Tensors with random values.

```
random_tensor = tf.random.uniform([2, 3], minval=0, maxval=10)
```

4. **Zeros and Ones:**
   - Tensors filled with zeros or ones.

```
zeros = tf.zeros([2, 3])

ones = tf.ones([2, 3])
```

Tensor Types

1. **0D Tensor (Scalar):**
   - A single value.
     scalar = tf.constant(42)
2. **1D Tensor (Vector):**
   - A list of values.
     vector = tf.constant([1, 2, 3])
3. **2D Tensor (Matrix):**
   - A table of values with rows and columns.
     matrix = tf.constant([[1, 2], [3, 4]])
4. **3D Tensor:**
   - An array of matrices (e.g., a batch of images).
     tensor_3d = tf.constant([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
5. **4D Tensor:**
   - An array of 3D tensors (e.g., a batch of videos).
     tensor_4d = tf.constant([[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]])
6. **Higher-Dimensional Tensors:**
   - Tensors with more dimensions can be created similarly.

Creating Operators

Operators in TensorFlow are functions that perform computations on tensors. Here are some basic operators:

Addition:

```
a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])
add = tf.add(a, b)
print(add)  # Output: [5 7 9]
```

Subtraction:

```
sub = tf.subtract(a, b)
print(sub)  # Output: [-3 -3 -3]
```

Multiplication:

```
mul = tf.multiply(a, b)
```

```
print(mul)  # Output: [ 4 10 18]
```

Division:

```
div = tf.divide(a, b)
print(div)  # Output: [0.25 0.4  0.5 ]
```

Matrix Multiplication:

```
matrix1 = tf.constant([[1, 2], [3, 4]])
matrix2 = tf.constant([[5, 6], [7, 8]])
matmul = tf.matmul(matrix1, matrix2)
print(matmul)  # Output: [[19 22]
               #         [43 50]]
```

IV.Executing Operators with Sessions

In TensorFlow 1.x, operations are not executed immediately. Instead, a computational graph is defined first, and then the operations are executed within a session. This approach allows TensorFlow to optimize the execution of the operations.

Key Concepts

1. **Computational Graph:**
   - A computational graph is a series of TensorFlow operations arranged into a graph of nodes. Each node represents an operation, and the edges represent the data (tensors) flowing between these operations.
2. **Session:**
   - A session encapsulates the control and state of the TensorFlow runtime. It is used to execute operations in the computational graph.

Steps to Execute Operators with Sessions

Import TensorFlow:

```
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
```

1. **Define the Computational Graph:**
   - Create tensors and define operations on them.

```
# Define constants
```

```
a = tf.constant(5)
b = tf.constant(3)

# Define an addition operation
add_op = tf.add(a, b)
```

2. **Create a Session:**
   - To execute the operations, you need to create a session.

```
with tf.Session() as sess:
    # Run the addition operation
    result = sess.run(add_op)
    print(result)  # Output: 8
```

Example: Adding Two Constants

Let's break down the code step-by-step:

1. **Import TensorFlow and Disable V2 Behavior:**
   - Since we are using TensorFlow 1.x syntax, we need to disable TensorFlow 2.x behavior.

```
import tensorflow.compat.v1 as tf
```

   - `tf.disable_v2_behavior()`

2. **Define Constants:**
   - Create two constant tensors, `a` and `b`.

```
a = tf.constant(5)
b = tf.constant(3)
```

3. **Define an Addition Operation:**
   - Create an addition operation `add_op` that adds the tensors `a` and `b`.

```
add_op = tf.add(a, b)
```

4. **Create and Run a Session:**
   - Create a session using `with tf.Session() as sess`.
   - Run the addition operation within the session and print the result.

```
with tf.Session() as sess:
    result = sess.run(add_op)
    print(result)  # Output: 8
```

Why Sessions Were Used:

1. **Explicit Graph Execution:**
    - TensorFlow operates on a computation graph where nodes represent operations and edges represent data flow (tensors). Sessions were used to instantiate and execute this graph.
2. **Resource Management:**
    - Sessions allowed TensorFlow to manage resources efficiently, such as GPU memory allocation. Sessions could be configured to run operations on specific hardware devices.
3. **Control Over Execution:**
    - Sessions provided explicit control over when and how computations were performed. This was crucial for handling distributed training across multiple devices or servers.

Evolution and Reasons for Removing Sessions:

1. **Eager Execution Introduced:**
    - TensorFlow introduced eager execution starting from version 2.0. Eager execution allows operations to be evaluated immediately, like regular Python code, without the need for constructing and running a computational graph within a session.
2. **Simplified Programming:**
    - Eager execution simplifies the programming model, making it easier for developers to debug models and write TensorFlow code more intuitively, akin to NumPy.
3. **Better Integration with Python:**
    - Eager execution seamlessly integrates with Python control flow and data structures, making TensorFlow more compatible with Python's ecosystem and enhancing usability.
4. **Dynamic Model Building:**
    - With eager execution, models can be built dynamically and inspected during runtime, facilitating experimentation and rapid prototyping.
5. **Improved Debugging:**
    - Errors are reported immediately during execution with eager mode, aiding in faster debugging and iteration cycles compared to deferred error reporting with sessions.

Sample Code with and without session:

TensorFlow 1.x Example (with Session)

```python
# Import TensorFlow 1.x
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()

# Define constants and operations
a = tf.constant(5)
b = tf.constant(3)
add_op = tf.add(a, b)

# Create a session
with tf.Session() as sess:
    # Run the operation within the session
    result = sess.run(add_op)
    print(f"Result with session: {result}")
```

TensorFlow 2.x Example (with Eager Execution)

```python
# Import TensorFlow 2.x (eager execution is enabled by default)
import tensorflow as tf

# Define constants and operations
a = tf.constant(5)
b = tf.constant(3)
add_result = tf.add(a, b)

# Print the result directly (no session needed)
print(f"Result with eager execution: {add_result.numpy()}")
```

Explanation

- **Session Usage**:
  - TensorFlow 1.x: Requires explicitly creating and managing sessions to execute operations within a defined computation graph.
  - TensorFlow 2.x: Eager execution allows for immediate execution of operations without the need for sessions.
- **Code Simplicity**:
  - TensorFlow 1.x: More boilerplate code is required for session management.
  - TensorFlow 2.x: Code is simplified, operations are evaluated like regular Python code, enhancing readability.

V. Understanding Code as a Graph and Setting Session Configurations

Understanding Code as a Graph

In TensorFlow 1.x, computations are represented as a dataflow graph, where:

- **Nodes** represent operations (e.g., addition, multiplication).
- **Edges** represent tensors (data arrays) flowing between these operations.

This computational graph is a directed graph where the nodes execute operations and the edges transport the output of one operation to the next.

Key Points:

- **Graph Construction**: Define operations and variables to build the computational graph.
- **Graph Execution**: Run operations within a session.

Example:

```
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()

# Define a simple graph
a = tf.constant(5)
b = tf.constant(3)
add_op = tf.add(a, b)

# This builds the graph but doesn't execute anything yet.
```

Summary

- **Code as a Graph**: TensorFlow 1.x represents computations as a graph of operations.
- **Session Configurations**: Sessions are used to execute the graph, with options for configuration to optimize execution and manage resources effectively.

Understanding these concepts is crucial for efficiently working with TensorFlow 1.x and leveraging its computational graph model.

VI. Writing Code in Jupyter, Using Variables, and Saving/Loading Variables

Writing Code in Jupyter

- **Jupyter Notebook**: An interactive environment that allows you to write and execute Python code in a cell-based format.
- **Code Cells**: Cells where you can write and execute TensorFlow code.
- **Markdown Cells**: Cells used for documentation and explanations.

Example:

```
import tensorflow as tf

# Define a TensorFlow constant
a = tf.constant(5)
b = tf.constant(3)
add_result = tf.add(a, b)

# Print the result
print(add_result.numpy())
```

Using Variables

- **Variables**: Used to store and update values during training or computation.
- **Creating Variables**: Use tf.Variable() to create a variable.
- **Updating Variables**: Variables can be updated using assignment operations.

Example:

```
# Define a variable
variable = tf.Variable(10)

# Update the variable
variable.assign(15)
print(variable.numpy())  # Output: 15
```

Saving Variables

- **Saving**: Use tf.saved_model.save() to save the entire model or tf.train.Checkpoint() for saving variables.

  Example:

```
# Define a variable
variable = tf.Variable(10)

# Save the variable
checkpoint = tf.train.Checkpoint(var=variable)
checkpoint.save('/path/to/save/model')
```

Loading Variables

- **Loading**: Use tf.train.Checkpoint.restore() to load saved variables.

  Example:

```
# Restore the variable
checkpoint = tf.train.Checkpoint(var=variable)
checkpoint.restore('/path/to/save/model-ckpt')
print(variable.numpy())  # Output: Restored value
```

Summary

- **Jupyter Notebooks**: Interactive environment for writing and executing code.
- **Variables**: Store and update values, created with tf.Variable().
- **Saving Variables**: Use tf.train.Checkpoint() to save and restore variable states.
- **Loading Variables**: Restore saved variables with tf.train.Checkpoint.restore().

The process for writing code, using variables, and saving/loading variables is similar in Google Colab. Google Colab is a cloud-based Jupyter notebook environment, so the principles and syntax are the same.

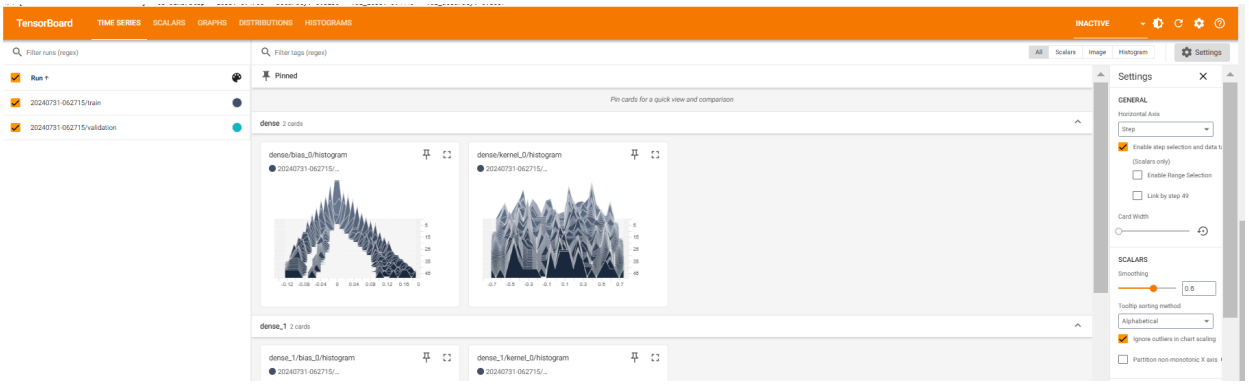
VII. Visualizing Data Using TensorBoard in Google Colab

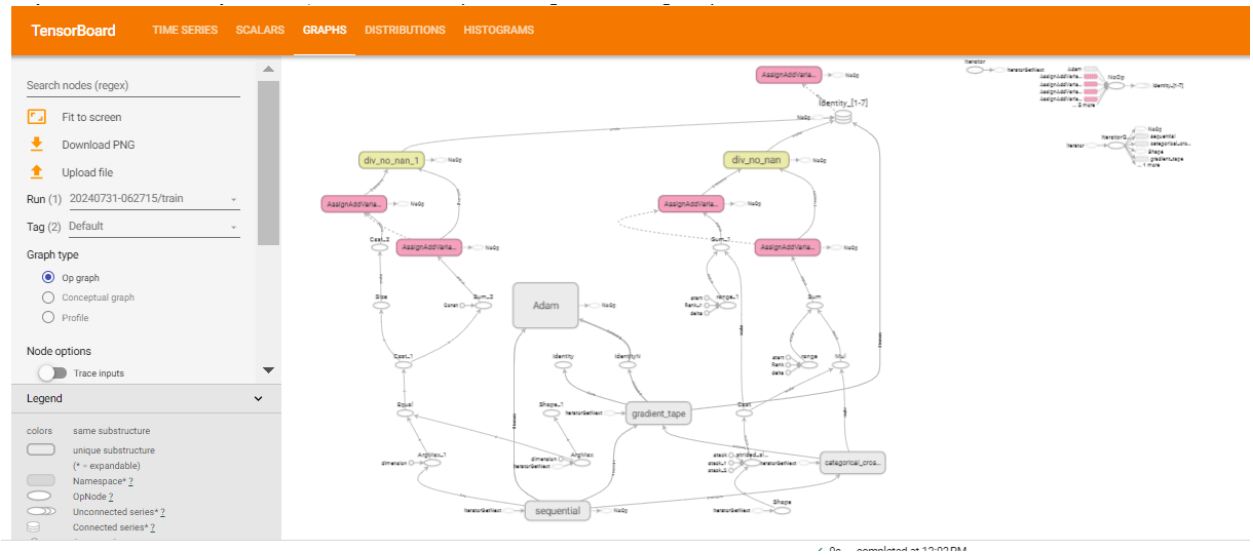
1. **Setup Logging**:
   - Create a summary writer with tf.summary.create_file_writer('logs').
   - Log data using tf.summary.scalar('name', value, step).
2. **Run Your Code**:
   - Execute the code to generate event files in the logs directory.
3. **Start TensorBoard**:

   In Colab, use:

   ```
   %load_ext tensorboard
   %tensorboard --logdir logs
   ```
4. **View Data**:
   - TensorBoard will display metrics and data in the "Scalars" tab.

Example Tensorboard Output: for colab notebook :
https://colab.research.google.com/drive/15W5XZCa6A2xz_XnjP6oOZmiT50hondGe





VIII. Overview of Keras

- **Keras**: A high-level API for building and training deep learning models. It provides a user-friendly interface to TensorFlow.
- **Features**:
    - **Simplified API**: Easy to use for beginners with fewer lines of code.
    - **Pre-built Layers**: Provides a wide range of layers and optimizers.
    - **Model Building**: Supports sequential and functional API for constructing models.

Architecture of Keras

Sequential API:

- **Description**: Models are created by stacking layers in a linear fashion.
- **Usage**: Ideal for simple, layer-by-layer models.

Functional API:

- **Description**: More flexible, allowing for complex architectures such as multi-input or multi-output models.
- **Usage**: Suitable for models with shared layers or complex layer connections.

Model Subclassing

- **Description:** Model Subclassing in Keras allows for creating custom models by inheriting from tf.keras.Model and defining your own call() method to specify the forward pass of the model.

- **Usage:**
  - **Custom Models**: Provides flexibility for complex architectures beyond Sequential and Functional APIs.
  - **Inheriting from tf.keras.Model**: Create a new model by subclassing the base Model class.
  - **Define __init__ and call Methods**:
    - **__init__**: Initialize layers and other components.
    - **call**: Define the forward pass of the model.

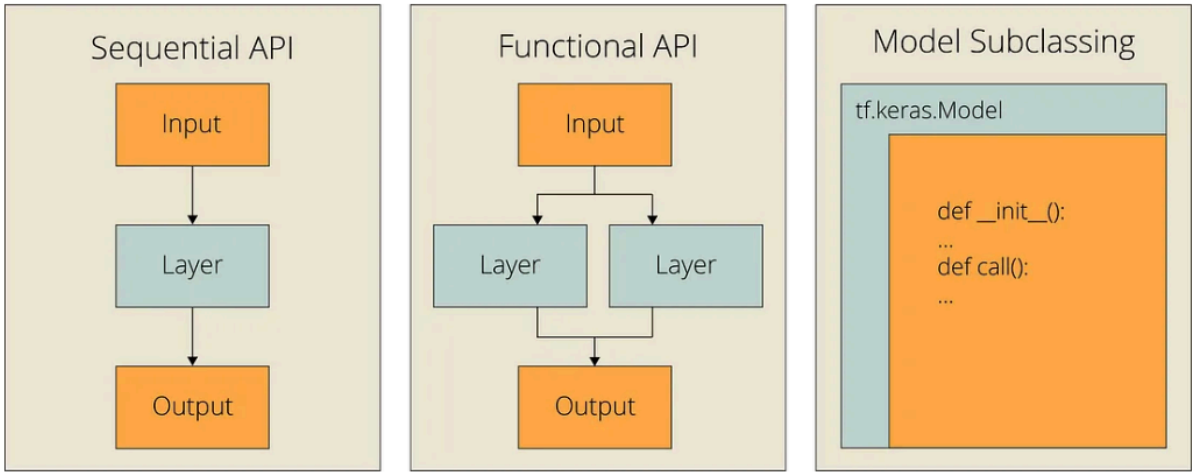


Figure 1. The Sequential API, The Functional API, Model Subclassing Methods Side-by-Side